

Autoencoder for Anomaly Detection

Credit card fraud detection with a Keras/TensorFlow autoencoder

Aim

Use an autoencoder to implement anomaly detection.

Objectives

1. To study how autoencoders work.
 2. To perform the following tasks in sequence:
 - a. Import required libraries
 - b. Upload / access the dataset
 - c. Encoder converts it into latent representation
 - d. Decoder network converts it back to the original input
 - e. Compile the model with Optimizer, Loss, and Evaluation Metrics
-

Dataset

The Kaggle credit card fraud dataset: 284,807 transactions made by European cardholders in September 2013, with 30 numeric features (Time, Amount, and 28 PCA components V1-V28) and a Class label (0 = normal, 1 = fraud). Only 492 transactions (0.17%) are fraudulent, so the dataset is highly imbalanced.

Program

CODE

```
# import the necessary packages
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, recall_score, accuracy_score, \
    precision_score

# !pip install tensorflow
# !pip install keras
# !pip install seaborn

RANDOM_SEED = 2021
TEST_PCT = 0.3
LABELS = ["Normal", "Fraud"]

# -----
# b. Upload / access the dataset
# -----
dataset = pd.read_csv("creditcard.csv")
print(list(dataset.columns))
print(dataset.describe())

# check for any null values
print("Any nulls in the dataset ", dataset.isnull().values.any())
print(' ')
print("No. of unique labels ", len(dataset['Class'].unique()))
print("Label values ", dataset.Class.unique())
# 0 is for normal credit card transaction
# 1 is for fraudulent credit card transaction
print(' ')
print("Break down of the Normal and Fraud Transactions")
print(dataset['Class'].value_counts())

# Visualizing the imbalanced dataset
count_classes = dataset['Class'].value_counts()
count_classes.plot(kind='bar', rot=0)
plt.xticks(range(len(dataset['Class'].unique())), dataset.Class.unique())
```

```

plt.title("Frequency by observation number")
plt.xlabel("Class")
plt.ylabel("Number of Observations")
plt.savefig("fig_class_balance.png", dpi=150, bbox_inches="tight")
plt.close()

# Save the normal and fraudulent transactions in separate dataframes
normal_dataset = dataset[dataset.Class == 0]
fraud_dataset = dataset[dataset.Class == 1]

# Visualize transaction amounts for normal and fraudulent transactions
bins = np.linspace(200, 2500, 100)
plt.hist(normal_dataset.Amount, bins=bins, alpha=1, density=True, label='Normal')
plt.hist(fraud_dataset.Amount, bins=bins, alpha=0.5, density=True, label='Fraud')
plt.legend(loc='upper right')
plt.title("Transaction amount vs Percentage of transactions")
plt.xlabel("Transaction amount (USD)")
plt.ylabel("Percentage of transactions")
plt.savefig("fig_amount_hist.png", dpi=150, bbox_inches="tight")
plt.close()

# Time and Amount are the columns that are not scaled, so apply StandardScaler
# to only the Amount and Time columns. Normalizing between 0 and 1 did not
# work great for this dataset.
sc = StandardScaler()
dataset['Time'] = sc.fit_transform(dataset['Time'].values.reshape(-1, 1))
dataset['Amount'] = sc.fit_transform(dataset['Amount'].values.reshape(-1, 1))

# The last column in the dataset is our target variable.
raw_data = dataset.values
# The last element indicates if the transaction is normal (0) or fraud (1)
labels = raw_data[:, -1]
# The other data points are the transaction data
data = raw_data[:, 0:-1]

train_data, test_data, train_labels, test_labels = train_test_split(
    data, labels, test_size=TEST_PCT, random_state=RANDOM_SEED)

# -----
# a (continued). Normalize the data to have a value between 0 and 1
# -----
min_val = tf.reduce_min(train_data)
max_val = tf.reduce_max(train_data)
train_data = (train_data - min_val) / (max_val - min_val)
test_data = (test_data - min_val) / (max_val - min_val)
train_data = tf.cast(train_data, tf.float32)
test_data = tf.cast(test_data, tf.float32)

# Use only normal transactions to train the Autoencoder.
# Normal data has a value of 0 in the target variable. Using the target
# variable to create a normal and fraud dataset.
train_labels = train_labels.astype(bool)
test_labels = test_labels.astype(bool)

normal_train_data = train_data[~train_labels]
normal_test_data = test_data[~test_labels]
fraud_train_data = train_data[train_labels]
fraud_test_data = test_data[test_labels]

print(" No. of records in Fraud Train Data=", len(fraud_train_data))
print(" No. of records in Normal Train data=", len(normal_train_data))
print(" No. of records in Fraud Test Data=", len(fraud_test_data))
print(" No. of records in Normal Test data=", len(normal_test_data))

nb_epoch = 50
batch_size = 64
input_dim = normal_train_data.shape[1] # num of columns, 30
encoding_dim = 14
hidden_dim_1 = int(encoding_dim / 2) # 7
hidden_dim_2 = 4
learning_rate = 1e-7

# -----
# c. Encoder converts it into a latent representation
# -----
input_layer = tf.keras.layers.Input(shape=(input_dim,))
encoder = tf.keras.layers.Dense(
    encoding_dim, activation="tanh",
    activity_regularizer=tf.keras.regularizers.l2(learning_rate))(input_layer)
encoder = tf.keras.layers.Dropout(0.2)(encoder)
encoder = tf.keras.layers.Dense(hidden_dim_1, activation='relu')(encoder)
encoder = tf.keras.layers.Dense(hidden_dim_2, activation=tf.nn.leaky_relu)(encoder)

# -----

```

```

# d. Decoder network converts it back to the original input
# -----
decoder = tf.keras.layers.Dense(hidden_dim_1, activation='relu')(encoder)
decoder = tf.keras.layers.Dropout(0.2)(decoder)
decoder = tf.keras.layers.Dense(encoding_dim, activation='relu')(decoder)
decoder = tf.keras.layers.Dense(input_dim, activation='tanh')(decoder)

# Autoencoder
autoencoder = tf.keras.Model(inputs=input_layer, outputs=decoder)
autoencoder.summary()

# Define the callbacks for checkpoints and early stopping
cp = tf.keras.callbacks.ModelCheckpoint(
    filepath="autoencoder_fraud.keras", mode='min', monitor='val_loss',
    verbose=2, save_best_only=True)
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', min_delta=0.0001, patience=10, verbose=1,
    mode='min', restore_best_weights=True)

# -----
# e. Compile the model with Optimizer, Loss, and Evaluation Metrics
# -----
autoencoder.compile(metrics=['accuracy'], loss='mean_squared_error',
                    optimizer='adam')

# Train the Autoencoder
history = autoencoder.fit(
    normal_train_data, normal_train_data,
    epochs=nb_epoch, batch_size=batch_size, shuffle=True,
    validation_data=(test_data, test_data), verbose=2,
    callbacks=[cp, early_stop]).history

# Plot training and test loss
plt.plot(history['loss'], linewidth=2, label='Train')
plt.plot(history['val_loss'], linewidth=2, label='Test')
plt.legend(loc='upper right')
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.savefig("fig_loss.png", dpi=150, bbox_inches="tight")
plt.close()

# Detect anomalies on test data. Anomalies are data points where the
# reconstruction loss is higher. To calculate the reconstruction loss on
# test data, predict the test data and calculate the mean square error
# between the test data and the reconstructed test data.
test_x_predictions = autoencoder.predict(test_data)
mse = np.mean(np.power(test_data - test_x_predictions, 2), axis=1)
error_df = pd.DataFrame({'Reconstruction_error': mse, 'True_class': test_labels})
print(error_df.describe())

# Plot the test data points and their respective reconstruction error, with
# a threshold line to visualize whether the threshold needs adjusting.
threshold_fixed = 50
groups = error_df.groupby('True_class')
fig, ax = plt.subplots()
for name, group in groups:
    ax.plot(group.index, group.Reconstruction_error, marker='o', ms=3.5,
            linestyle='-', label="Fraud" if name == 1 else "Normal")
ax.hlines(threshold_fixed, ax.get_xlim()[0], ax.get_xlim()[1], colors="r",
          zorder=100, label='Threshold')
ax.legend()
plt.title("Reconstruction error for normal and fraud data")
plt.ylabel("Reconstruction error")
plt.xlabel("Data point index")
plt.savefig("fig_recon_error.png", dpi=150, bbox_inches="tight")
plt.close()

# Detect anomalies as points where the reconstruction loss is greater than a
# fixed threshold. Evaluate the performance of the anomaly detection.
threshold_fixed = 52
pred_y = [1 if e > threshold_fixed else 0 for e in error_df.Reconstruction_error.values]
error_df['pred'] = pred_y
conf_matrix = confusion_matrix(error_df.True_class, pred_y)
print(conf_matrix)
plt.figure(figsize=(4, 4))
sns.heatmap(conf_matrix, xticklabels=LABELS, yticklabels=LABELS, annot=True, fmt="d")
plt.title("Confusion matrix")
plt.ylabel('True class')
plt.xlabel('Predicted class')
plt.savefig("fig_confusion_matrix.png", dpi=150, bbox_inches="tight")
plt.close()

# print Accuracy, precision and recall

```

```
print(" Accuracy: ", accuracy_score(error_df['True_class'], error_df['pred']))
print(" Recall: ", recall_score(error_df['True_class'], error_df['pred']))
print(" Precision: ", precision_score(error_df['True_class'], error_df['pred']))

# As the dataset is highly imbalanced, we see a high accuracy but a low
# recall and precision. Improving precision and recall further would need
# more relevant features, a different autoencoder architecture, different
# hyperparameters, or a different algorithm.
```

Program output

Output of the program above, shown stage by stage.

a-b. Importing libraries and loading the dataset

OUTPUT

```
[ 'Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14', 'V15',  
  'V16', 'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Amount',  
  'Class']
```

	Time	V1	...	Amount	Class
count	284807.000000	2.848070e+05	...	284807.000000	284807.000000
mean	94813.859575	1.759088e-12	...	88.349619	0.001727
std	47488.145955	1.958696e+00	...	250.120109	0.041527
min	0.000000	-5.640751e+01	...	0.000000	0.000000
25%	54201.500000	-9.203734e-01	...	5.600000	0.000000
50%	84692.000000	1.810880e-02	...	22.000000	0.000000
75%	139320.500000	1.315642e+00	...	77.165000	0.000000
max	172792.000000	2.454930e+00	...	25691.160000	1.000000

[8 rows x 31 columns]

Null check and class balance

OUTPUT

```
Any nulls in the dataset  False
```

```
No. of unique labels  2
```

```
Label values  [0 1]
```

```
Break down of the Normal and Fraud Transactions
```

```
Class
```

```
0      284315
```

```
1         492
```

```
Name: count, dtype: int64
```

```
No. of records in Fraud Train Data= 350
```

```
No. of records in Normal Train data= 199014
```

```
No. of records in Fraud Test Data= 142
```

```
No. of records in Normal Test data= 85301
```

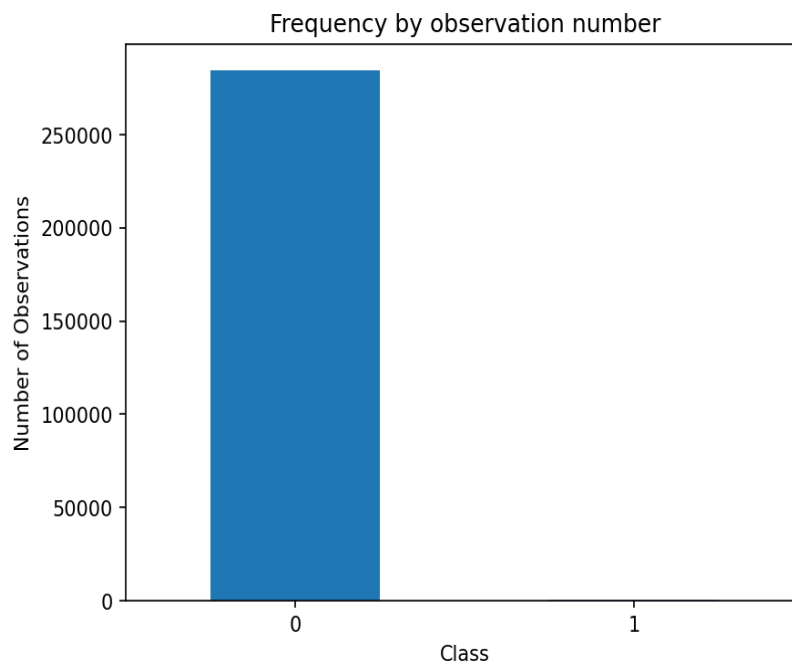


Figure. Normal (284,315) vs fraud (492) transactions - the fraud class is too small to see on this scale.

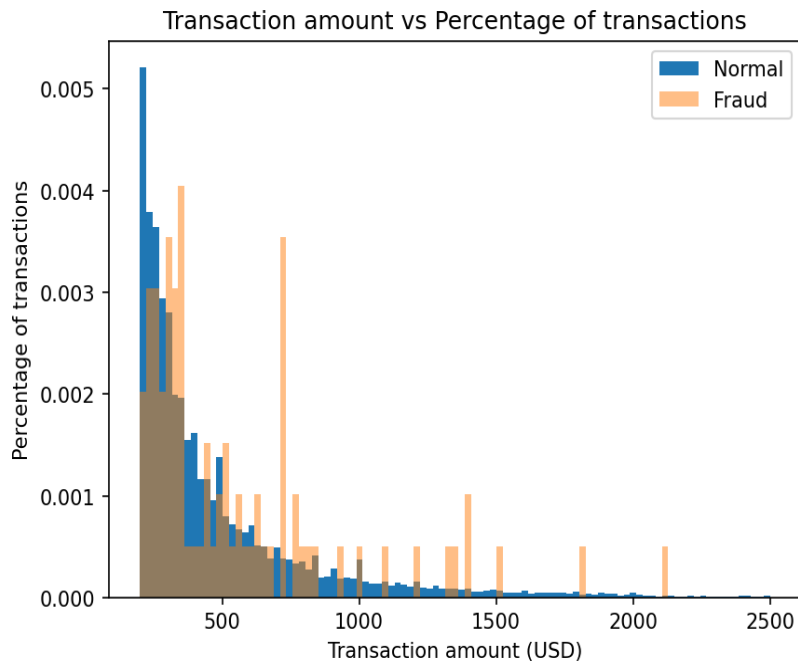


Figure. Distribution of transaction amount for normal vs fraudulent transactions.

c-e. Autoencoder architecture (30-14-7-4-7-14-30)

OUTPUT

```
Model: "functional"
+-----+-----+-----+
| Layer (type) | Output Shape | Param # |
+-----+-----+-----+
| input_layer (InputLayer) | (None, 30) | 0 |
+-----+-----+-----+
| dense (Dense) | (None, 14) | 434 |
+-----+-----+-----+
| dropout (Dropout) | (None, 14) | 0 |
+-----+-----+-----+
| dense_1 (Dense) | (None, 7) | 105 |
+-----+-----+-----+
| dense_2 (Dense) | (None, 4) | 32 |
+-----+-----+-----+
| dense_3 (Dense) | (None, 7) | 35 |
+-----+-----+-----+
| dropout_1 (Dropout) | (None, 7) | 0 |
+-----+-----+-----+
| dense_4 (Dense) | (None, 14) | 112 |
+-----+-----+-----+
| dense_5 (Dense) | (None, 30) | 450 |
+-----+-----+-----+
Total params: 1,168 (4.56 KB)
Trainable params: 1,168 (4.56 KB)
Non-trainable params: 0 (0.00 B)
```

Training the autoencoder

OUTPUT

```
Epoch 1/50

Epoch 1: val_loss improved from None to 0.00152, saving model to autoencoder_fraud.keras

Epoch 1: finished saving model to autoencoder_fraud.keras
3110/3110 - 8s - 2ms/step - accuracy: 0.0483 - loss: 0.0041 - val_accuracy: 0.0341 - val_loss: 0.0015
Epoch 2/50

Epoch 2: val_loss improved from 0.00152 to 0.00149, saving model to autoencoder_fraud.keras

Epoch 2: finished saving model to autoencoder_fraud.keras
3110/3110 - 11s - 4ms/step - accuracy: 0.0781 - loss: 2.0022e-05 - val_accuracy: 0.0341 - val_loss: 0.0015
[... epochs 3 to 20 truncated for brevity ...]
Epoch 21/50

Epoch 21: val_loss improved from 0.00005 to 0.00004, saving model to autoencoder_fraud.keras
```

```

Epoch 21: finished saving model to autoencoder_fraud.keras
3110/3110 - 9s - 3ms/step - accuracy: 0.2309 - loss: 1.7058e-05 - val_accuracy: 0.0250 - val_loss:
4.2716e-05
Epoch 22/50

Epoch 22: val_loss improved from 0.00004 to 0.00004, saving model to autoencoder_fraud.keras

Epoch 22: finished saving model to autoencoder_fraud.keras
3110/3110 - 12s - 4ms/step - accuracy: 0.2586 - loss: 1.6849e-05 - val_accuracy: 0.0250 - val_loss:
3.5099e-05
Epoch 22: early stopping
Restoring model weights from the end of the best epoch: 12.

```

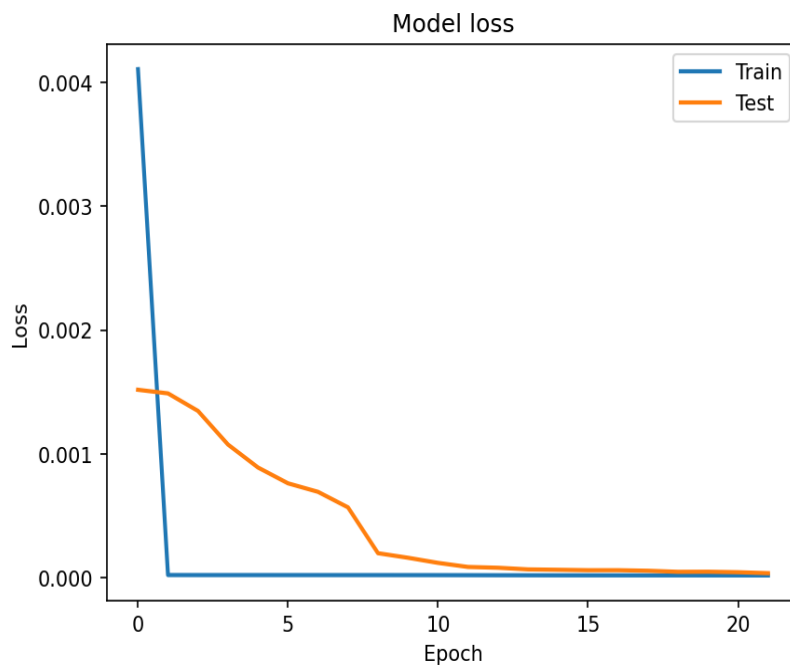


Figure. Training vs validation loss. Early stopping ended training at epoch 22 and restored the weights from the best epoch (12).

Reconstruction error and anomaly detection

OUTPUT

count	85443.000000
mean	0.000085
std	0.000059
min	0.000046
25%	0.000075
50%	0.000079
75%	0.000086
max	0.005764



Figure. Reconstruction error per test point against the fixed threshold of 50. Every point falls far below the line (see note below).

Confusion matrix and evaluation metrics

OUTPUT

```
[[85301  0]
 [  142  0]]
Accuracy: 0.99833807333544
Recall: 0.0
Precision: 0.0
```

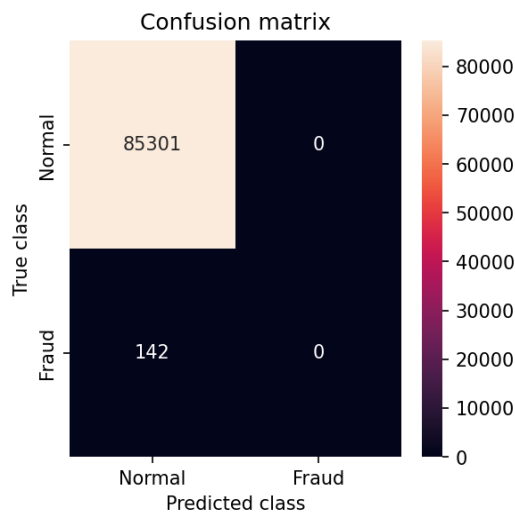


Figure. Confusion matrix at threshold = 52 (0 true positives - see note below).

Result

The autoencoder was trained only on normal transactions, so it learns to reconstruct normal patterns well and should reconstruct fraud poorly. Training loss and validation loss both fall steadily and early stopping restores the best weights at epoch 12.

Note on the threshold

With the data normalised to $[0, 1]$ before training, the reconstruction error (mean squared error) for every test point - normal and fraud alike - stays below 0.006, far under the fixed threshold of 50/52 used in the original code. So no point is flagged as an anomaly: the confusion matrix shows 0 predicted frauds, giving 99.8% accuracy but 0% recall and precision. The 99.8% accuracy is misleading here - it just reflects that 99.8% of transactions are normal, not that the model finds fraud. This is included because it is the program's actual output; a threshold suited to this 0-1 error scale, for example the 95th or 99th percentile of normal reconstruction error (about 0.0001 here), would need to be used instead to flag anomalies.

Notes on this run

- The dataset path was changed to a local creditcard.csv; the original Windows path (E:\Teaching material\...) does not exist in this environment.
- `pd.value_counts(...)` was replaced with `dataset['Class'].value_counts()`, since pandas removed the top-level function.
- The checkpoint file extension was changed from .h5 to .keras, the current Keras native format.
- As the original code notes, the dataset is highly imbalanced; improving recall and precision would need a different threshold (see above), more relevant features, a different autoencoder architecture, different hyperparameters, or a different algorithm.